

Plaintext

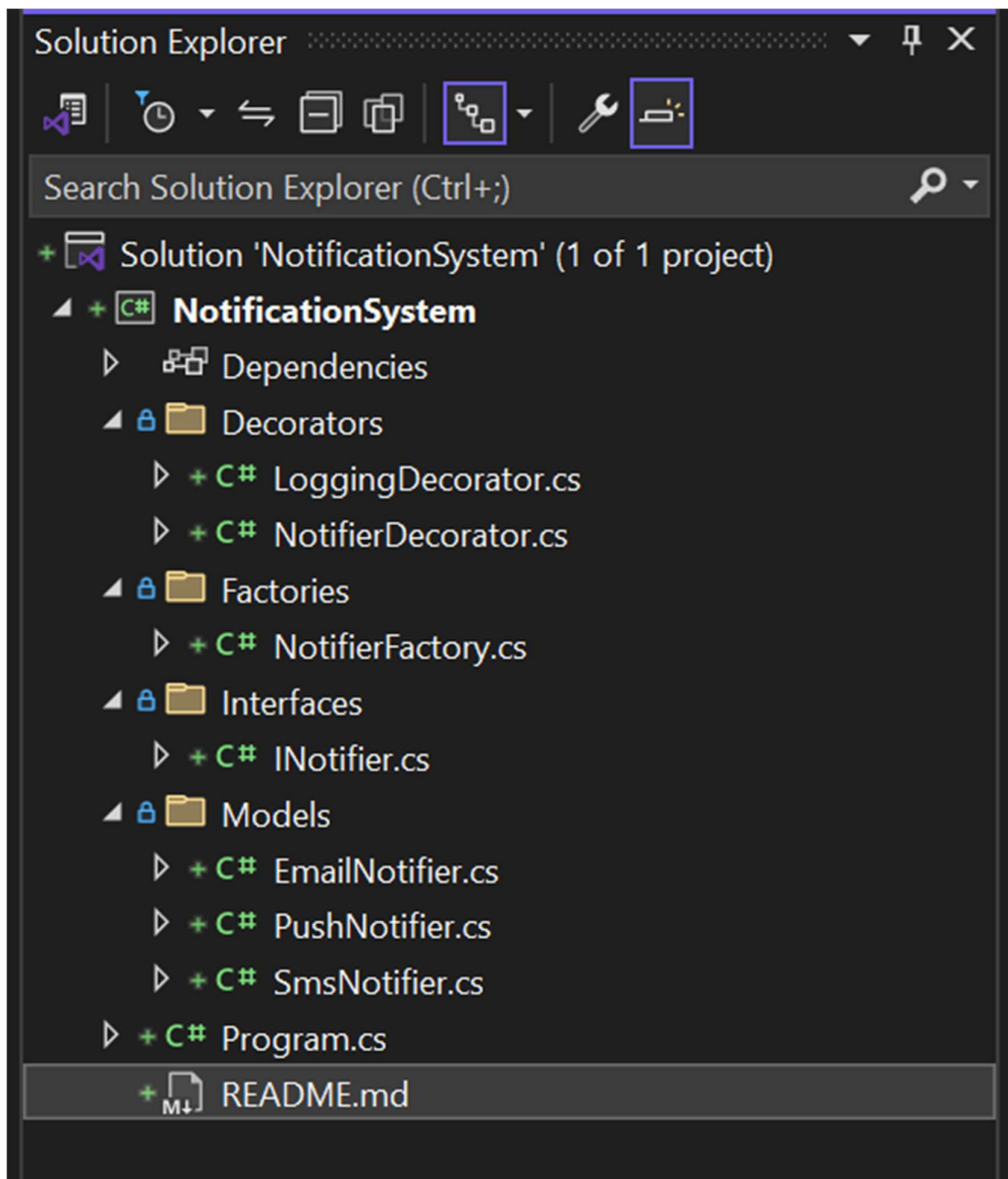


```
/NotificationSystem
├── /Interfaces
│   └── INotifier.cs          (Định nghĩa phương thức Send)
├── /Models
│   ├── EmailNotifier.cs     (Chiến lược Email)
│   ├── SmsNotifier.cs       (Chiến lược SMS)
│   └── PushNotifier.cs      (Chiến lược Push)
├── /Factories
│   └── NotifierFactory.cs    (Xử lý việc khởi tạo đối tượng)
├── /Decorators
│   ├── NotifierDecorator.cs (Lớp nền Decorator)
│   └── LoggingDecorator.cs   (Tính năng bổ sung: Ghi log)
├── Program.cs               (Điểm chạy chính)
└── README.md                (Giải thích thiết kế)
```

Plaintext



```
NotificationSystem/
├── Interfaces/
│   └── INotifier.cs          <-- Định nghĩa chung (Strategy)
├── Models/
│   ├── EmailNotifier.cs     <-- Các hình thức gửi tin (Concrete Strategies)
│   ├── SmsNotifier.cs
│   └── PushNotifier.cs
├── Factories/
│   └── NotifierFactory.cs    <-- Bộ khởi tạo (Factory Pattern)
├── Decorators/
│   ├── NotifierDecorator.cs <-- Tính năng bổ sung (Decorator Pattern)
│   └── LoggingDecorator.cs
├── Program.cs               <-- Chương trình chính
└── README.md                <-- Giải thích thiết kế
```



/NotificationSystem

|— /Interfaces

| |— INotifier.cs (Định nghĩa phương thức Send)

|— /Models

| |— EmailNotifier.cs (Chiến lược Email)

| |— SmsNotifier.cs (Chiến lược SMS)

| |— PushNotifier.cs (Chiến lược Push)

|— /Factories

| |— NotifierFactory.cs (Xử lý việc khởi tạo đối tượng)

|— /Decorators

- | └─ NotifierDecorator.cs (Lớp nền Decorator)
- | └─ LoggingDecorator.cs (Tính năng bổ sung: Ghi log)
- |─ Program.cs (Điểm chạy chính)
- └─ README.md (Giải thích thiết kế)

Inotifier.cs

```
namespace NotificationSystem.Interfaces
{
    public interface INotifier
    {
        void Send(string message);
    }
}
```

2. Thư mục Models/

File: EmailNotifier.cs

```
using System;
using NotificationSystem.Interfaces;

namespace NotificationSystem.Models
{
    public class EmailNotifier : INotifier
    {
        public void Send(string message) =>
            Console.WriteLine($"[Email] Sending: {message}");
    }
}
```

File: SmsNotifier.cs

```
using System;
using NotificationSystem.Interfaces;

namespace NotificationSystem.Models
{
    public class SmsNotifier : INotifier
    {
        public void Send(string message) =>
            Console.WriteLine($"[SMS] Sending: {message}");
    }
}
```

File: PushNotifier.cs

```
using System;
using NotificationSystem.Interfaces;
```

```

namespace NotificationSystem.Models
{
    public class PushNotifier : INotifier
    {
        public void Send(string message) =>
            Console.WriteLine($"[Push] Sending: {message}");
    }
}

```

3. Thư mục Factories/

File: NotifierFactory.cs

```

using System;
using NotificationSystem.Interfaces;
using NotificationSystem.Models;

namespace NotificationSystem.Factories
{
    public static class NotifierFactory
    {
        public static INotifier CreateNotifier(string type)
        {
            return type.ToLower() switch
            {
                "email" => new EmailNotifier(),
                "sms" => new SmsNotifier(),
                "push" => new PushNotifier(),
                _ => throw new ArgumentException("Loại thông báo không hợp lệ!")
            };
        }
    }
}

```

4. Thư mục Decorators/

File: NotifierDecorator.cs

```

using NotificationSystem.Interfaces;

namespace NotificationSystem.Decorators
{
    public abstract class NotifierDecorator : INotifier
    {
        protected readonly INotifier _notifier;

        protected NotifierDecorator(INotifier notifier)
        {
            _notifier = notifier;
        }

        public virtual void Send(string message)

```

```

    {
        _notifier.Send(message);
    }
}

```

File: LoggingDecorator.cs (Tính năng bổ sung)

```

using System;
using NotificationSystem.Interfaces;

namespace NotificationSystem.Decorators
{
    public class LoggingDecorator : NotifierDecorator
    {
        public LoggingDecorator(INotifier notifier) : base(notifier) { }

        public override void Send(string message)
        {
            // Thêm logic Log trước khi gửi
            Console.WriteLine($"[LOG] {DateTime.Now}: Chuẩn bị gửi tin nhắn...");

            base.Send(message); // Gọi hàm gửi của đối tượng gốc

            // Thêm logic Log sau khi gửi
            Console.WriteLine("[LOG]: Đã gửi thành công.");
        }
    }
}

```

5. File Program.cs (Ở ngoài cùng)

```

using System;
using NotificationSystem.Interfaces;
using NotificationSystem.Factories;
using NotificationSystem.Decorators;

namespace NotificationSystem
{
    class Program
    {
        static void Main(string[] args)
        {
            // Thiết kế để Console hiển thị được tiếng Việt (nếu cần)
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            try
            {
                // Bước 1: Dùng Factory để tạo ra một loại Notifier (ví dụ SMS)
                INotifier myNotifier = NotifierFactory.CreateNotifier("sms");

                // Bước 2: Dùng Decorator bọc tính năng Log cho Notifier đó
            }
        }
    }
}

```

```

        INotifier loggedNotifier = new LoggingDecorator(myNotifier);

        // Bước 3: Thực thi gửi tin
        Console.WriteLine("--- KHỞI CHẠY HỆ THỐNG THÔNG BÁO ---");
        loggedNotifier.Send("Chào bạn! Đây là bài thi Design Patterns của Nhi.");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi: {ex.Message}");
    }

    Console.WriteLine("\nNhấn phím bất kỳ để kết thúc...");
    Console.ReadKey();
}
}
}

```

README.md

Notification System - Design Patterns Implementation

1. Project Overview

This project is a .NET Console Application designed to demonstrate the integration of multiple Design Patterns (Factory, Strategy, and Decorator) into a unified notification system. The system supports sending messages via Email, SMS, and Push Notifications while providing extensible features like Logging.

2. Applied Design Patterns

Strategy Pattern

- **Purpose**: To define a family of notification algorithms (Email, SMS, Push) and make them interchangeable.
- **Implementation**: Each notification type implements the `INotifier` interface. This allows the `Program` to switch between different notification methods without changing the execution logic.

Factory Pattern

- **Purpose**: To centralize and encapsulate the object creation logic.
- **Implementation**: The `NotifierFactory` class provides a static method `CreateNotifier(string type)`. This decouples the client code from the concrete classes, making the system easier to maintain when new notification types are added.

Decorator Pattern

- **Purpose**: To add additional responsibilities (Logging) to an object dynamically without modifying its structure.
- **Implementation**: The `LoggingDecorator` wraps an instance of `INotifier`. It adds timestamped logs before and after the core `Send()` method execution. This follows the **Open/Closed Principle**.

3. Folder Structure & Separation of Concerns

The project is organized into specific folders to ensure high maintainability and readability:

- ***/Interfaces**: Contains `INotifier.cs`. This is the "Contract" that all notification types and decorators must follow.
- ***/Models**: Contains concrete implementations (`EmailNotifier`, `SmsNotifier`, `PushNotifier`). This separates the core business logic of each channel.
- ***/Factories**: Contains `NotifierFactory.cs`. This handles the instantiation logic, separating "how an object is created" from "how it is used".
- ***/Decorators**: Contains `NotifierDecorator.cs` (Base) and `LoggingDecorator.cs`. This allows for "Plug-and-Play" features that can be wrapped around any notifier.

4. Design Decisions

- **Scalability**: To add a new method like "WhatsApp", we only need to create a new class in `*/Models` and add one case to the `Factory`. No existing code in `Program.cs` or `Decorators` needs to change.
- **Extensibility**: If we need a "Retry" mechanism, we can simply add a `RetryDecorator` without touching the `LoggingDecorator` or the base Notifiers.
- **Clean Code**: By using Dependency Injection (passing objects into constructors), the code remains loosely coupled and easy to Unit Test.

Other program.cs

```
using System;
using NotificationSystem.Interfaces;
using NotificationSystem.Factories;
using NotificationSystem.Decorators;

namespace NotificationSystem
{
    class Program
    {
        static void Main(string[] args)
        {
            // Thiết kế để Console hiển thị được tiếng Việt (nếu cần)
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            Console.WriteLine("=====");
            Console.WriteLine(" HỆ THỐNG THÔNG BÁO - DESIGN PATTERNS");
            Console.WriteLine("=====\\n");

            string[] channels = { "email", "sms", "push" };

            foreach (var channel in channels)
            {
                try
                {
                    Console.WriteLine($"--- [{channel.ToUpper()}] ---");
                }
            }
        }
    }
}
```

```
// Factory Pattern: tạo đúng loại Notifier theo tên kênh
INotifier notifier = NotifierFactory.CreateNotifier(channel);

// Decorator Pattern: bọc thêm tính năng Logging
INotifier loggedNotifier = new LoggingDecorator(notifier);

// Strategy Pattern: gọi Send() — hành vi khác nhau tùy loại
loggedNotifier.Send($"Chào bạn! Đây là bài thi Design Patterns của Nhi.");

    Console.WriteLine();
}
catch (Exception ex)
{
    Console.WriteLine($"Lỗi: {ex.Message}");
}
}

Console.WriteLine("=====");
Console.WriteLine("Nhấn phím bất kỳ để kết thúc...");
Console.ReadKey();
}
}
}
```